

SEB-Naver: A $SE(2)$ -based Local Navigation Framework for Car-like Robots on Uneven Terrain

Xiaoying Li^{†2}, Long Xu^{†1,2}, Xiaolin Huang^{†2}, Donglai Xue², Zhihao Zhang²,
Zhichao Han^{1,2}, Chao Xu^{1,2}, Yanjun Cao², and Fei Gao^{1,2}

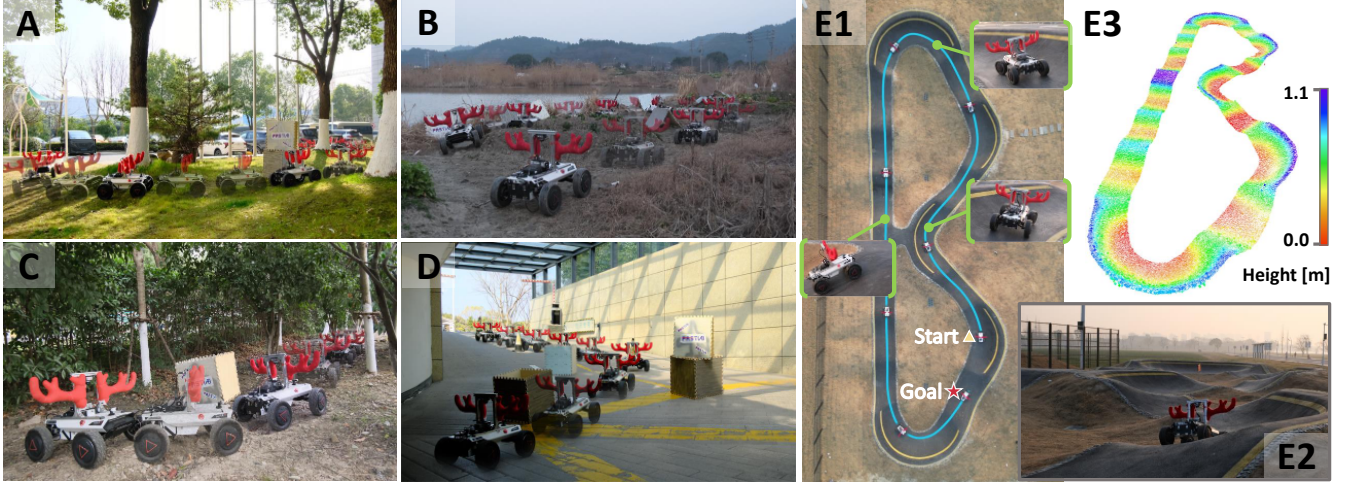


Fig. 1: SEB-Naver on the grasslands (A), farmland (B), forests (C), underground parking lots (D), and pump track (E).

Abstract—Autonomous navigation of car-like robots on uneven terrain poses unique challenges compared to flat terrain, particularly in *traversability assessment* and *terrain-associated kinematic* modelling for motion planning. This paper introduces SEB-Naver, a novel $SE(2)$ -based local navigation framework designed to overcome these challenges. First, we propose an efficient traversability assessment method for $SE(2)$ grids, leveraging GPU parallel computing to enable real-time updates and maintenance of local maps. Second, inspired by differential flatness, we present an optimization-based trajectory planning method that integrates terrain-associated kinematic models, significantly improving both planning efficiency and trajectory quality. Finally, we unify these components into SEB-Naver, achieving real-time terrain assessment and trajectory optimization. Extensive simulations and real-world experiments demonstrate the effectiveness and efficiency of our approach. The code is at https://github.com/ZJU-FAST-Lab/seb_naver.

I. INTRODUCTION

Autonomous navigation of car-like robots on uneven terrain requires additional considerations compared to flat terrain, specifically for *traversability assessment* and *terrain-associated kinematic model*. The former is an additional module integrated into the autonomous navigation framework, which estimates the risk of anomalies based on the

terrain and state of the robot. The latter is a factor that must be considered in the trajectory generation module to ensure that the planned trajectory can be well tracked by the robot.

Classical methods [1]–[4] for *traversability assessment* tend to focus only on the geometric information of the terrain, such as slope, curvature, etc. To achieve real-time updates in local navigation frameworks, they usually construct robot-centric grid maps by discretizing the $\mathbb{R}^2$ space, leading to conservative or aggressive results. As shown in Fig. 2 (a), different robot poses with varying pitch and roll angles may be projected onto the same grid cell in $\mathbb{R}^2$ space. However, since the tolerance of the robot to lateral tilt and backward tilt differs, the risk values should also vary. A more detailed evaluation of traversability based on the robot state in $SE(3)$ often incurs a significantly higher computational burden [5]. Moreover, due to the substantial increase in spatial dimensions, using discrete grids to achieve real-time updates becomes impractical.

In our previous work [6], we propose *terrain pose mapping* to describe the impact of terrain on the robot, allowing us to derive the $SE(3)$ state of the robot from a given state in $SE(2)$. However, the $SE(2)$ space remains too large to meet real-time requirements. In this work, we draw inspiration from parallel computing, proposing an efficient traversability assessment method for $SE(2)$ grids using GPU, achieving real-time updates and maintenance of the local map.

Fig. 2 (b) illustrates how the kinematic model of the robot is influenced by the terrain. Due to gravity, the throttle

[†]Indicates equal contribution.

¹State Key Laboratory of Industrial Control Technology, Zhejiang University, Hangzhou 310027, China. *Corresponding author: Fei Gao*

²Huzhou Institute of Zhejiang University, Huzhou 313000, China.

E-mail: {gaolon, fgaoaa}@zju.edu.cn

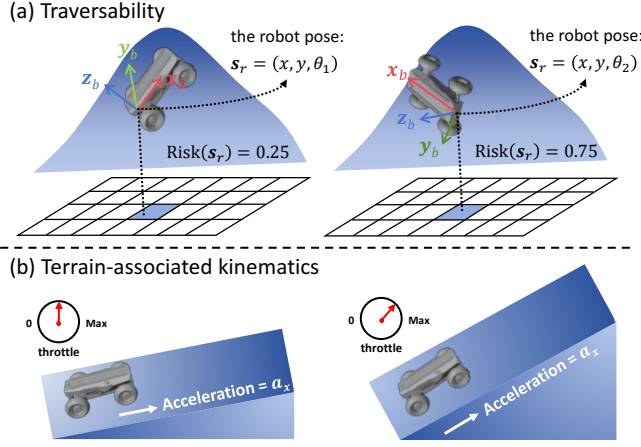


Fig. 2: Figure (a) illustrates that different robot poses with different pitch and roll angles may be projected onto the same grid cell in $\mathbb{R}^2$. Figure (b) illustrates that the throttle required for the robot to achieve the same acceleration a_x is different due to the presence of gravity.

required to achieve the same acceleration in the body frame varies with the slope. If the planner ignores this factor, the generated trajectory may become infeasible for the robot. Therefore, the *terrain-associated kinematic model* must be considered by the trajectory planner to enable the controller to better track the trajectory. Due to the nonlinearity of the terrain and the high-dimensional robot state space, which increases the dimensionality of the problem, most existing works [5]–[8] struggle to effectively incorporate *terrain-associated kinematic model* into planning, making it challenging to balance efficiency and trajectory quality.

Differential flatness has demonstrated its efficiency in trajectory planning for autonomous driving [9], [10]. In this paper, we find that although the presence of uneven terrain complicates the kinematics, the state of the robot can still be represented using flat outputs through *terrain pose mapping*, simplifying the optimization problem. Furthermore, our derivations reveal that the *terrain-associated kinematic constraints* can also be analytically modeled by flat outputs. Based on this, we propose a novel optimization-based trajectory planning method for car-like robots on uneven terrain, significantly improving efficiency and trajectory quality. Moreover, we integrate it with the designed real-time traversability assessment algorithm for $SE(2)$ grids, proposing SEB-Naver, a $SE(2)$ -based local navigation framework. Extensive simulations and real-world experiments validate the efficiency and effectiveness of our approach. The contributions of the paper are:

- 1) We propose a local mapping pipeline that facilitates real-time traversability assessment for $SE(2)$ grids by utilizing the parallel computing power of GPU.
- 2) Based on differential flatness, we propose a novel trajectory optimization model that can effectively consider *terrain-associated kinematics* and significantly improve the efficiency of car-like robots on uneven terrain.

- 3) Integrating the above two modules, we propose SEB-Naver, a $SE(2)$ -based local navigation framework. Extensive simulations and real-world experiments verify the practicality and efficiency of our approach.

II. RELATED WORKS

A. LiDAR-based Traversability Assessment

Traversability assessment is a critical component for safe and robust navigation on rough terrain. Based on LiDAR, most of the existing works [1]–[4] primarily focus on the geometric features of the terrain, such as ground height, curvature, slope, etc. To meet real-time requirements, these approaches construct and maintain a robot-centric grid map by discretizing the $\mathbb{R}^2$ space, ensuring efficiency. However, this reliance on geometric features can lead to overly conservative or aggressive terrain evaluation results. For instance, different robot poses with varying pitch and roll angles may be projected onto the same grid cell in $\mathbb{R}^2$ when the evaluation is centred on the rear axle of a car-like robot, as shown in Fig. 2 (a). To address the issue of overly conservative or aggressive evaluation results, some works [5], [6], [8] estimate traversability in $SE(3)$ space. This allows for a more detailed consideration of pose-related variables, such as the pitch and roll angles of the robot, as well as suspension [11]. However, due to the high dimensionality of the $SE(3)$ space (locally homeomorphic to $\mathbb{R}^6$), most methods [5], [8] tend to perform the evaluation when traversability values are required, which may significantly impact the efficiency of trajectory planning.

In our previous work [6], *terrain pose mapping* was proposed to enable the construction of grid maps in the lower-dimensional $SE(2)$ space for traversability assessment, improving the efficiency of trajectory generation. However, the number of grids generated by discretizing $SE(2)$ remains relatively large, making it insufficient to support real-time updates, as realized with local elevation maps [1]. In this paper, we leverage the powerful parallel computing capabilities of GPUs to design a local mapping framework in $SE(2)$ space, achieving real-time performance.

On the other hand, more techniques, such as neural networks [12]–[15] and the Gaussian process, have recently been applied to traversability assessment. Some works [16]–[19] employ self-supervised learning methods to compute traversability maps, which have been demonstrated to be effective for autonomous navigation in unstructured outdoor environments. The main limitation of these learning-based approaches lies in the uncertainty of the models, which can significantly affect the reliability of the evaluation [20]. Other works [21]–[23] model traversability assessment using the Gaussian process, enhancing spatial continuity compared to classical elevation maps. Nevertheless, this approach suffers from low computational efficiency and struggles to effectively integrate historical point clouds from LiDAR.

B. Motion Planning on Uneven Terrain

In recent years, researchers have extended 2D planning methods to enable car-like robots to navigate uneven terrain

autonomously. Krusi et al. [5] proposed a trajectory planning method based on unordered 3D point clouds obtained from LiDAR, incorporating concurrent terrain assessment into the planning process, but it struggles with real-time performance due to computational limitations. Han et al. [24] proposed a 2D dynamic sampling method for uneven terrain, incorporating a real-time elevation map with physics-based constraints. This approach leverages the model predictive path integral (MPPI) algorithm alongside a low-level controller to enable high-speed robot navigation on challenging terrain. However, dense sampling in complex environments limits its efficiency.

To enhance planning efficiency, Jian et al. [8] proposed a hybrid architecture combining plane-fitting, informed-RRT* path searching, Gaussian process regression, and NMPC-based trajectory generation. But its computational scalability is limited by the exponentially increasing complexity of NMPC formulations as the planning horizon extends. To address this issue, Xu et al. [6] proposed a polynomial-based trajectory optimization framework, incorporating equality constraints to handle the nonholonomic constraints of the robot. While this method improves planning efficiency, the function values corresponding to the equality constraints become very small when the robot starts moving or shifts gears, making it difficult for the optimizer to converge. This not only reduces efficiency but also makes it challenging to adapt their method to scenarios where the robot needs to frequently alternate between forward and backward movements. Inspired by the differential flatness of car-like robots [9], [10], we find that flat outputs still exist for *terrain-associated kinematics*. Based on this, we propose a new trajectory optimization model which can avoid the introduction of the equational constraint. Besides, we introduce an intermediate variable to eliminate the singularities associated with differential flatness, enabling more efficient planning.

III. TERRAIN POSE MAPPING

In this paper, we use the left superscript to denote the frame in which the vector is located, and ${}^B_A \mathbf{R}$ to denote the rotation matrix from frame A to frame B . We use S to denote the LiDAR frame, B to denote the robot body frame, and M to denote the map frame. Without labelling, the world frame W is used by default.

On flat terrain, the state of a car-like robot can be represented using $\mathbf{s}_r = [x, y, \theta]^T \in SE(2)$. While on uneven terrain, due to the variation in terrain height, we need to represent the state of the robot using a higher dimensional $SE(3)$ state with the position $\mathbf{p}_B = [x, y, z]^T \in \mathbb{R}^3$ and the attitude ${}_B \mathbf{R} = [\mathbf{x}_b, \mathbf{y}_b, \mathbf{z}_b] \in SO(3)$.

In our previous work [6], we proposed *terrain pose mapping* $\mathcal{F} : SE(2) \mapsto \mathbb{R} \times \mathbb{S}_+^2$, where $\mathbb{S}_+^2 \triangleq \{\mathbf{x} \in \mathbb{R}^3 \mid \|\mathbf{x}\|_2 = 1, \mathbf{b}_3^T \mathbf{x} > 0\}$, $\mathbf{b}_3 = [0, 0, 1]^T$ to describe the impact of terrain on the robot. Let $\mathbf{x}_{yaw} = [\cos \theta, \sin \theta, 0]^T$ denote the direction of yaw angle, the mapping $\mathcal{F}$ can be expressed as two functions $z = f_1(x, y, \theta)$, $\mathbf{z}_b = \mathbf{f}_2(x, y, \theta)$. Using the Z-X-Y Euler angles to represent the attitude of the

robot, we can obtain:

$$\mathbf{p}_B = [x, y, z]^T = [x, y, f_1(x, y, \theta)]^T, \quad (1)$$

$$\mathbf{y}_b = \frac{\mathbf{f}_2(x, y, \theta) \times \mathbf{x}_{yaw}}{\|\mathbf{f}_2(x, y, \theta) \times \mathbf{x}_{yaw}\|}, \quad (2)$$

$$\mathbf{x}_b = \mathbf{y}_b \times \mathbf{f}_2(x, y, \theta). \quad (3)$$

Thus, with the help of the mapping $\mathcal{F}$, we can still use elements in $SE(2)$ to characterize the state of the robot on uneven terrain. The mapping $\mathcal{F}$ actually implies the height and attitude given by the uneven terrain to $\mathbf{s}_r$.

IV. OVERVIEW OF THE SEB-NAVER

The local navigation problem for car-like robots on uneven terrain can be described as how to obtain a strategy $\pi \sim p(\mathbf{u}_t | \mathbf{O}_t, \mathbf{c}_t)$ such that the robot can successfully reach the destination and satisfy certain conditions during the process, where $\mathbf{O}_t$ denotes past observations, $\mathbf{c}_t \in SE(2)$ denotes the current target pose, and $\mathbf{u}_t$ denotes the control command to send to the robot. The conditions will typically include 1) the risk of robot failure is as small as possible, 2) control commands can be tracked well by the robot, and 3) the robot can move fast while conserving energy. We decompose the problem into several modules, as shown in Fig. 3.

To handle the first condition, we construct a local map and consider the risk in the local trajectory planning module. In this work, we use the pose information from the localization module and the point clouds from LiDAR as observations $\mathbf{O}_t$ to construct a risk map. We mainly consider the second and third conditions in trajectory optimization, which specifically correspond to the kinematic constraints of the robot and the objective function containing smoothness and agility. For these two conditions to be better satisfied, we include an MPC controller downstream of the planner that outputs control commands $\mathbf{u}_t$ including steering angle δ and velocity v_x at a higher frequency. It is only used to track the optimized trajectory without considering obstacle avoidance, risk, etc.

V. MAPPING IN $SE(2)$ SPACE BASED ON GPU

Fig. 4 illustrates the proposed process of local mapping in $SE(2)$ space. When receiving new pose information and point cloud, the GPU filters the point cloud and computes their variance in parallel. It also resets some elevation grids by ray casting. At the same time, grids outside the range of the map are removed in parallel due to the movement of the map caused by changes in the position of the robot. After these, the elevation map will be updated by Kalman filtering (KF) and can be inpainted by classical methods [25], [26] or neural network (NN) based approaches [27], [28]. The GPU will compute the traversability for each $SE(2)$ state in parallel using the inpainted elevation map. Eventually, the computed risk map will be sent back to the CPU, and the corresponding signed distance field [29] (SDF) will be generated to be used for collision avoidance.

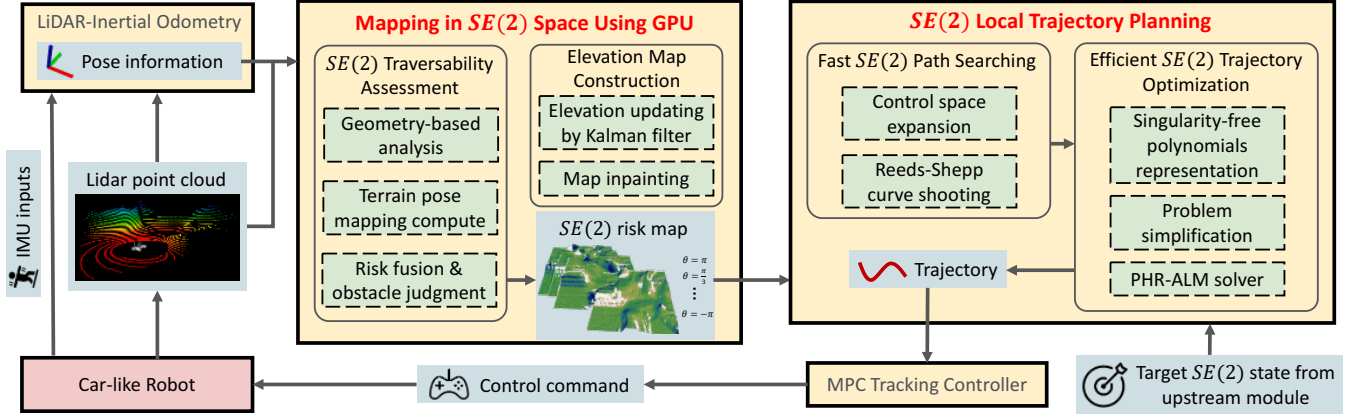


Fig. 3: Overview of the framework. LiDAR-Inertial Odometry (LIO) receives information from IMU and point clouds from LiDAR to calculate pose. Then, the local mapping module acquires the point cloud and poses information to update the elevation and evaluate the traversability in $SE(2)$ space. The resulting risk maps are used by the local planner. After path search and trajectory optimization, the local planner outputs a trajectory in $SE(2)$ space, which is given to the MPC controller for generating final control commands to send to the robot.

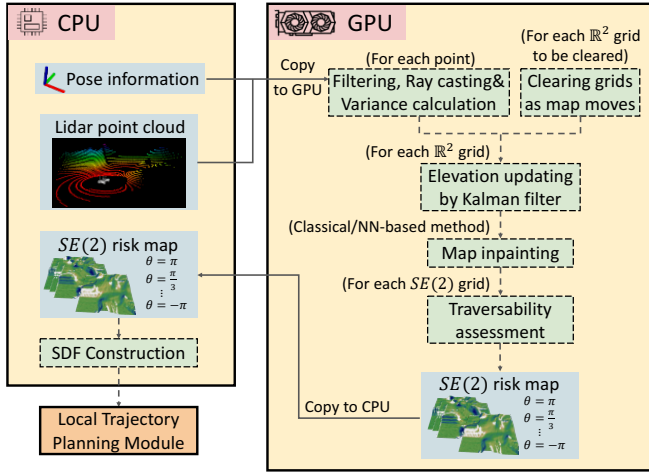


Fig. 4: Local mapping process in $SE(2)$ space.

A. Elevation Map Construction

In this paper, we set ${}_M\mathbf{R} = \mathbf{I}$, where $\mathbf{I}$ denotes the identity matrix. While the position of the map frame,

$$\mathbf{p}_M = l_{\text{res}} \cdot [\text{floor}(x/l_{\text{res}}), \text{floor}(y/l_{\text{res}}), 0]^T, \quad (4)$$

is set to be determined by the robot position, where function $\text{floor}(q)$ serves to take the largest integer not greater than q . After acquiring the robot pose and LiDAR point cloud, we first update $\mathbf{p}_M$ according to Eq.(4), and then set the state of grids that are no longer within the map to unknown. Besides, we apply parallel ray casting, checking the grid that passes from the LiDAR position to each point, setting the state to unknown if the elevation in the grid is higher than the highest value predicted by the ray.

For points in the point cloud, we ignore the points outside the range of the map. Then, we transform the others to the robot body frame B and filter out the points outside a set height range. The remaining points will be assigned to

the corresponding grids depending on the range each grid includes. Let one point be $\mathbf{p}_l = [x_l, y_l, z_l]^T$. Notice that

$$\begin{aligned} z_l &= \mathbf{b}_3^T \mathbf{p}_l = \mathbf{b}_3^T ({}_S\mathbf{R}^S \mathbf{p}_l + \mathbf{p}_S) \\ &= \mathbf{b}_3^T ({}_B\mathbf{R}({}_S\mathbf{R}^S \mathbf{p}_l + {}^B\mathbf{p}_{BS}) - \mathbf{p}_B), \end{aligned} \quad (5)$$

where $\mathbf{p}_S$ denotes the position of the LiDAR, ${}^B\mathbf{p}_{BS}$ denotes the position of the LiDAR in the robot body frame, ${}_S\mathbf{p}_l, {}_B\mathbf{R}, \mathbf{p}_B$ are random variables. We can then compute the Jacobi of z_l as follows:

$$\mathbf{J}_S = \frac{\partial z_l}{\partial {}^S\mathbf{p}_l} = ({}_B\mathbf{R}_S^B \mathbf{R})^T \mathbf{b}_3, \quad (6)$$

$$\mathbf{J}_R = \frac{\partial z_l}{\partial {}_B\mathbf{R}} = ({}_S\mathbf{R}^S \mathbf{p}_l + {}^B\mathbf{p}_{BS})^\wedge {}_B\mathbf{R}^T \mathbf{b}_3, \quad (7)$$

$$\mathbf{J}_B = \frac{\partial z_l}{\partial \mathbf{p}_B} = -\mathbf{b}_3, \quad (8)$$

where the operations $\mathbf{q}^\wedge$ takes the vector $\mathbf{q} \in \mathbb{R}^3$ to form a skew-symmetric matrix. We can obtain the variance of z_l :

$$\sigma_{z_l}^2 = \mathbf{J}_S^T \Sigma_S \mathbf{J}_S + \mathbf{J}_R^T \Sigma_R \mathbf{J}_R + \mathbf{J}_B^T \Sigma_B \mathbf{J}_B, \quad (9)$$

where $\Sigma_S, \Sigma_R, \Sigma_B$ are covariance matrices. Σ_S is determined by the sensor model, while Σ_R, Σ_B are given by the localization module.

After calculating the variances for all points, we update the elevations of the $\mathbb{R}^2$ grids in parallel using a one-dimensional Kalman filter and handle multiple points that fall into the same grids by Mahalanobis distance-based method, in the same way as the classical elevation map [1].

B. Traversability Assessment

We propose Algorithm 1 to evaluate the risk and *terrain pose mapping* in state $\mathbf{s}_r$. Similar to our previous work [6], we take the points corresponding to the elevation map in an elliptical area related to the robot's size, obtaining z_b and the approximate terrain curvature κ_{ter} by geometry-based analysis. After using the resultant z_b to calculate

Algorithm 1: Traversability and *terrain pose mapping* assessment in state $s_r \in SE(2)$

Input: elevation map $\mathcal{M}$, state $s_r \in SE(2)$, parameters of ellipse (e_x, e_y) , weights $w_r \in \mathbb{R}^3$, Constants $\kappa_{\max}, \varphi_{x\max}, \varphi_{y\max}$

Output: Risk(s_r), $z_b(s_r)$

begin

```

 $\mathcal{P} \leftarrow \text{FindEllipticalPoints}(\mathcal{M}, s_r, e_x, e_y);$ 
 $p_{\text{mean}} \leftarrow \text{GetMeanPosition}(\mathcal{P});$ 
 $Cov \leftarrow \text{ZeroMatrix3}();$ 
for each  $p_j \in \mathcal{P}$  do
     $p_e \leftarrow p_j - p_{\text{mean}};$ 
     $Cov \leftarrow Cov + p_e p_e^T;$ 
 $Cov \leftarrow Cov / \text{NumOf}(\mathcal{P});$ 
 $z_b, \kappa_{\text{ter}} \leftarrow \text{GetMinEigenVecWithCurv}(Cov);$ 
if  $\kappa_{\text{ter}} > \kappa_{\max}$  then
    return  $(1, z_b).$ 
 $x_b, y_b \leftarrow \text{GetXbYb}(z_b, s_r);$ 
 $\varphi_x \leftarrow |\arcsin(b_3^T x_b)|, \varphi_y \leftarrow |\arcsin(b_3^T y_b)|;$ 
if  $\varphi_x > \varphi_{x\max} \vee \varphi_y > \varphi_{y\max}$  then
    return  $(1, z_b).$ 
 $r \leftarrow \left[ \frac{\kappa_{\text{ter}}}{\kappa_{\max}}, \frac{\varphi_x}{\varphi_{x\max}}, \frac{\varphi_y}{\varphi_{y\max}} \right]^T;$ 
Risk( $s_r$ )  $\leftarrow r^T w_r;$ 
return (Risk( $s_r$ ),  $z_b$ ).

```

y_b and x_b by Eq.(2) and Eq.(3), we check the angle of these two vectors with the horizontal plane to determine the traversability, respectively. We fuse the curvature κ_{ter} , representing the geometric information of the terrain, and the angles φ_x and φ_y , representing the attitude of the robot, by weighted summation to obtain the final risk. Regions with Risk(s_r) = 1 are judged as explicit obstacles.

VI. LOCAL TRAJECTORY PLANNING

A. Trajectory Parameterization

The kinematic equations for car-like robot driving on uneven terrain can be represented as follows:

$$\dot{p}_B = v_x \cdot x_b, \quad (10)$$

$${}_B \dot{R} = {}_B R \left[\frac{v_x \tan \delta}{L_w} \cdot z_b \right], \quad (11)$$

where L_w is the wheelbase length of the robot, the operation $[*]$ takes a vector to a skew-symmetric matrix. In our previous work, we demonstrated the use of *terrain pose mapping* to reduce the dimensionality of the robot to $SE(2)$ space, as well as the efficiency of trajectory planning brought by polynomial parameterizations. To satisfy the nonholonomic constraints of the robots, it introduces the following equational constraint in the trajectory optimization modelling:

$$\dot{x}(t) \sin \theta(t) - \dot{y}(t) \cos \theta(t) = 0. \quad (12)$$

However, given that $v_x(t) = \eta(t) \sqrt{\dot{x}^2(t) + \dot{y}^2(t)}$, the left-hand side of Eq.(12) becomes very small when the robot's

speed is low (e.g., when the robot starts moving or changes gears), where $\eta(t) = 1 \vee -1$ denotes the robot is moving forward or backward. This leads to numerical issues in the optimization process, which not only reduces efficiency but also makes it difficult to extend the method to scenarios where the robot needs to frequently alternate between moving forward and backward. To address this problem, we draw inspiration from the differential flatness of car-like robots [9], [10], introducing an intermediate variable $s(t)$ to represent the trajectory.

Reconsidering Eq.(12), if there is $\dot{x}^2(t) + \dot{y}^2(t) \neq 0$, we can explicitly compute $\theta(t) = \arctan2(\eta(t)\dot{y}(t), \eta(t)\dot{x}(t))$. So we can rewrite Eq.(12) with $s(t)$ as:

$$\dot{x}(s)\dot{s}(t) \sin \theta(t) + \dot{y}(s)\dot{s}(t) \cos \theta(t) = 0. \quad (13)$$

Thus, we can add the constraint $\dot{x}^2(s) + \dot{y}^2(s) > \delta_+$, obtaining $\theta(t) = \arctan2(\eta(t)\dot{y}(s), \eta(t)\dot{x}(s))$, eliminating the equation constraint (12), where $\delta_+ > 0$ is a constant. This approach avoids the singularity of the function $\arctan2(*)$ in Eq.(12) when $\dot{x}^2(t) + \dot{y}^2(t) = 0$.

Similar to the work [6], we use quintic piecewise polynomials with four times continuously differentiable at the segmented points to represent state trajectories. Each piece of trajectory can be denoted as:

$$x_i(s) = c_{x_i}^T \beta(s), \quad s \in [0, s(T_p) - s(0)], \quad (14)$$

$$y_i(s) = c_{y_i}^T \beta(s), \quad s \in [0, s(T_p) - s(0)], \quad (15)$$

$$s_i(t) = c_{s_i}^T \beta(t), \quad t \in [0, T_p], \quad (16)$$

where $i = 1, 2, \dots, N$ is the index of piecewise polynomial, T_p is the duration of one piece of the trajectory, $c_* \in \mathbb{R}^6$, $*$ = $\{x_i, y_i, s_i\}$ is the coefficient of polynomial, $\beta(t) = [1, t, t^2, \dots, t^5]^T$ is the natural base.

Using the *terrain pose mapping* and the state transfer equations Eq.(10) and (11), letting $y_{yaw} = [-\sin \theta, \cos \theta, 0]^T$, we can derive the control inputs and some kinematic variables from the parameterized trajectory as follows:

$$v_x = \frac{\dot{s}(t) \sqrt{\dot{x}^2(s) + \dot{y}^2(s)}}{x_b^T x_{yaw}}, \quad (17)$$

$$\omega_z = \frac{\dot{x}(s)\ddot{y}(s) - \dot{y}(s)\ddot{x}(s)}{(\dot{x}^2(s) + \dot{y}^2(s))z_b^T b_3} \cdot \dot{s}(t) \quad (18)$$

$$\delta = \arctan2(L_w \omega_z, v_x) \quad (19)$$

$$a_x = \frac{\dot{s}^2(t) v_{ds}^T a_{ds} + \ddot{s}(t) v_{ds}^T v_{ds}}{\sqrt{\dot{x}^2(s) + \dot{y}^2(s)} x_b^T x_{yaw}} + g \cdot x_b^T b_3, \quad (20)$$

$$a_y = \frac{\dot{x}(s)\ddot{y}(s) - \dot{y}(s)\ddot{x}(s)}{\sqrt{\dot{x}^2(s) + \dot{y}^2(s)} y_b^T y_{yaw}} \dot{s}^2(t) + g \cdot y_b^T b_3, \quad (21)$$

where $v_{ds} = [\dot{x}(s), \dot{y}(s)]^T$, $a_{ds} = [\ddot{x}(s), \ddot{y}(s)]^T$. a_x, a_y denote longitude and latitude acceleration, respectively.

B. Optimization Problem

In this paper, we formulate the local trajectory optimization problem for Car-like robots on uneven terrain as:

$$\min_{\mathbf{c}, \mathbf{e}_m, T_f} f(\mathbf{c}, \mathbf{e}_m, T_f) = \int_0^{T_f} \mathbf{j}(t)^T \mathbf{j}(t) dt + \rho_t T_f + \rho_r \int_0^{T_f} \text{Risk}^2(\mathbf{s}_r(t)) dt \quad (22)$$

$$s.t. \quad \mathbf{M}_p(\mathbf{S})\mathbf{c}_p = \mathbf{b}_p(\mathbf{P}, \mathbf{e}_m), \quad (23)$$

$$\mathbf{M}_s(T_f)\mathbf{c}_s = \mathbf{b}_s(\mathbf{S}), \quad (24)$$

$$T_f > 0, \quad \dot{x}^2(s) + \dot{y}^2(s) \geq \delta_+, \quad (25)$$

$$\delta^2(t) \leq \delta_{\max}, \quad v_x^2(t) \leq v_{\text{mlon}}^2, \quad (26)$$

$$a_x^2(t) \leq a_{\text{mlon}}^2, \quad a_y^2(t) \leq a_{\text{mlat}}^2, \quad (27)$$

$$\varphi_x^2(t) \leq \varphi_{x\max}^2, \quad \varphi_y^2(t) \leq \varphi_{y\max}^2, \quad (28)$$

$$\text{Risk}(\mathbf{s}_r(t)) \leq r_{\max}, \quad \mathcal{G}_c(\mathbf{s}_r(t)) \geq d_{\min}, \quad (29)$$

where $\mathbf{c}_p = [[\mathbf{c}_{x_1}, \mathbf{c}_{y_1}]^T, [\mathbf{c}_{x_2}, \mathbf{c}_{y_2}]^T, \dots, [\mathbf{c}_{x_N}, \mathbf{c}_{y_N}]^T]^T \in \mathbb{R}^{6N \times 2}$, $\mathbf{c}_s = [\mathbf{c}_{s_1}^T, \mathbf{c}_{s_2}^T, \dots, \mathbf{c}_{s_N}^T]^T \in \mathbb{R}^{6N \times 1}$ are coefficient matrices. $\mathbf{c} = \{\mathbf{c}_p, \mathbf{c}_s\}$. For the other optimization variables, $\mathbf{e}_m \in \mathbb{R}^{4 \times \Omega}$, where each column of it is a boundary variable $\mathbf{e}_m^w = [x(s_w), y(s_w), \dot{x}(s_w), \dot{y}(s_w)]$, $w = 1, 2, \dots, \Omega$, denoting the moment point at the w^{th} gear switch. T_f denote the total duration of the trajectory. In the objective function (22), $\mathbf{j}(t) = [x^{(3)}(t), y^{(3)}(t)]^T$ denote the jerk of the trajectories. $\rho_t > 0, \rho_r > 0$ are constants to give the trajectory some aggressiveness and reduce risk, respectively. Eq.(23) and Eq.(24) denote the combinations of the continuity constraints for polynomials and boundary conditions of the trajectory.

$\mathbf{P} \in \mathbb{R}^{2 \times (N-1)}$ are the segment points of the trajectories, $\mathbf{S} \in \mathbb{R}^{N \times 1}$ denote the value of each segment of the intermediate variable. Thus $\mathbf{M}_p, \mathbf{M}_s \in \mathbb{R}^{(6N-2) \times 6N}$. In practice, we set $\dot{x}^2(0) + \dot{y}^2(0) = 1$ and $\delta_+ = 0.9$ for optimization.

Conditions (25) denote the positive duration and the constraints brought by the introduction of $s(t)$ mentioned in the last subsection. Conditions (26) and (27) are dynamic feasibility constraints, including limitation of steering angle $\delta(t)$, velocity $v_x(t)$, longitude acceleration $a_x(t)$, and latitude acceleration $a_y(t)$, where $\delta_{\max}, v_{\text{mlon}}, a_{\text{mlon}}, a_{\text{mlat}}$ are constants. Conditions (28) are used to ensure that the robot's attitude remains within a safe range.

Conditions (29) represent safety constraints, ensuring the risk stays below the threshold $r_{\max}$ and avoiding explicit obstacles. To enforce this, we use the SDF $\mathcal{G}_c(\mathbf{s}_r) : SE(2) \mapsto \mathbb{R}$, where value at each state is the distance to the edge of the nearest region, with negative values inside obstacles. $d_{\min}$ defines the obstacle avoidance threshold.

C. Problem Solving

To address the equality constraints (23) and (24), we refer to the method proposed in work [30], adding the constraints of the trajectories having zero second-order derivatives at the start, termination and gear switch time to ensure that the matrices $\mathbf{M}_p, \mathbf{M}_s$ are invertible. This allows us to transform the optimization variables into $\{\mathbf{P}, \mathbf{S}, \mathbf{e}_m, T_f\}$, while simultaneously reducing the dimensionality of the optimization variables.

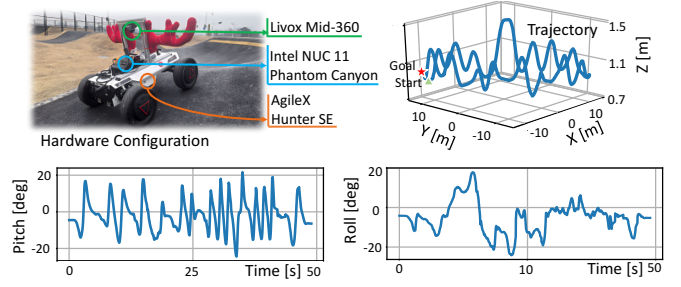


Fig. 5: One test of the SEB-Naver on the pump track.

To deal with the constraint $T_f > 0$, we convert the optimization variable T_f to τ using the logarithmic function: $\tau = \ln(T_f)$. For the remaining inequality constraints, we discretize each duration $T_p = T_f/N$ as K time stamps $\tilde{t}_p = (p/K) \cdot T_p, p = 0, 1, \dots, K-1$, and impose the constraints at these timestamps. We also discretize $\text{Risk}(\mathbf{s}_r(t))$ in this way and accumulate it to obtain its integral. For conditions (29), we adopt trilinear interpolation to obtain the values and gradients of the function $\mathcal{G}_c$ and Risk, where operations on manifold [31] are applied for the handling of $SO(2)$ space.

Then, the Powell-Hestenes-Rockafellar augmented Lagrangian method [32] (PHR-ALM), an effective approach for solving large-scale problems and problems with complex constraints, is utilized to address the simplified trajectory optimization problem. To obtain the initial solution, we employ a lightweight hybrid-A* algorithm to perform path planning on a $SE(2)$ grid map. The Reed-Shepp curve [33] is used to shoot toward the goal state, allowing the search process to terminate early.

VII. EXPERIMENTS

A. Implementation Details

We adopt CUDA [34] to implement the proposed real-time traversability assessment pipeline for $SE(2)$ grids, further deploying SEB-Naver on a car-like robot for real-world experiments, as shown in Fig. 5. FAST-LIO2 [35] is used for localization. We also build simulation environments based on EPFL terrain generator¹ to conduct comparative experiments with existing methods. All of them are run on a desktop with an Intel i5-14600 CPU and Nvidia GeForce RTX1660 GPU.

B. Comparison Experiments

1) *Mapping in $SE(2)$* : We compared the computation time of the proposed local mapping method with the baseline (CPU) [6] on two devices: the above-mentioned desktop computer and an edge computing device, Jetson Xavier NX. To analyze the computation time under different grid resolutions, we set the dimensions of the $\mathbb{R}^2$ space to range from $8m \times 8m$ to $18m \times 18m$ with a resolution of $0.1m \times 0.1m$, and the number of grids in the $SO(2)$ space varied between 8 and 32. For the proposed method, the processing includes all features shown in Fig. 4: point cloud filtering, ray casting, variance computing, height update and

¹<https://github.com/droduit/procedural-terrain-generation>

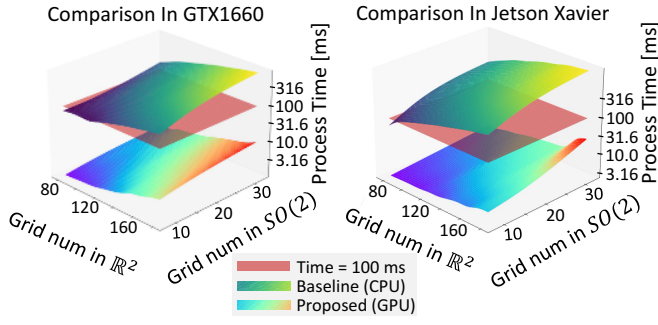


Fig. 6: Processing time comparison of $SE(2)$ grids.

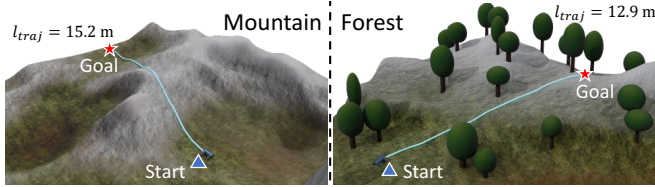


Fig. 7: Two examples of benchmark scenarios. l_{traj} denotes the length of the trajectory generated by SEB-Naver.

traversability assessment. The baseline method has the same pipeline except for ray casting. Both methods utilize nearest-neighbor interpolation for elevation inpainting.

The results are presented in Fig. 6. Since our processing is done on GPU, the processing time remained short even for large numbers of grids. Specifically, our method processes the densest grid configurations of 972,000 grids within 50 milliseconds, meeting the real-time requirement set by the frequency of 20 Hz. On the other hand, the baseline method requires over 100 milliseconds in most cases. This shows that our mapping approach is more efficient than the baseline in handling grids in $SE(2)$ space despite the additional ray-casting computations performed in our pipeline.

2) *Trajectory planning*: We compared the proposed planner with Xu’s [6] and Jian’s [8] methods in the terrains generated by the terrain generator mentioned before. Fig. 7 shows two examples of uneven mountainous and forested. The steering angle, velocity, longitude acceleration, and latitude acceleration of the car-like robot are limited to $\delta_{\max} = 0.785$ rad, $v_{\max} = 1.0$ m/s, $a_{\max} = 5.0$ m/s², $a_{\max} = 10.0$ m/s², respectively. The attitude limitations of the robot are set to $\varphi_{x\max} = \varphi_{y\max} = 0.52$ rad. We first conducted a qualitative comparison, as shown in Table I. Since both the proposed method and Xu’s [6] method use continuous polynomials rather than discrete states for trajectory parameterization, they exhibit better continuity and can provide higher-order acceleration information for the robot. To evaluate the impact of supporting reversing, we further compared the velocity curve with Xu’s [6] method on an uneven snowy mountain terrain. As shown in Fig. 8, our approach expands the solution space by considering the possibility of negative longitudinal velocities of the robot, which results in shorter trajectory duration and length. This is a huge advantage when robots are navigating on complex, uneven terrain.

We also conducted benchmark comparisons in 20 sim-

TABLE I: Methods Qualitative Comparison

Method	Reversing	Acceleration information	Continuity
Proposed	Support	✓	4-order
Xu’s [6]	No support	✓	4-order
Jian’s [8]	Support	✗	1-order

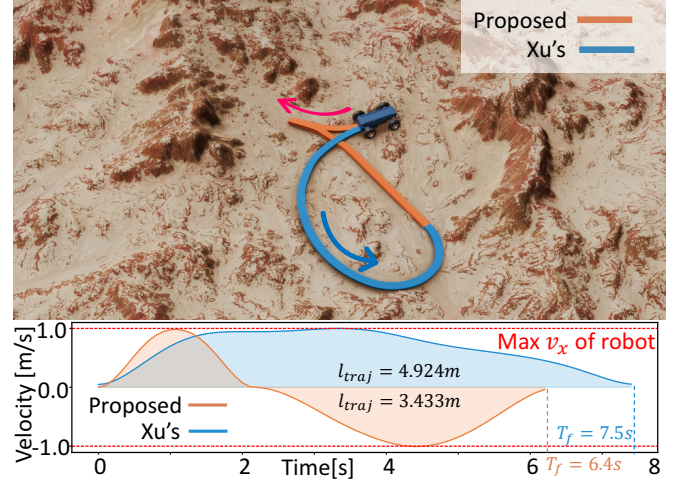


Fig. 8: Comparison of velocity curves on a snowy mountain.

ulated environments similar to those in Fig. 7. A total of two hundred starts and goals are sampled in each of these scenarios. The results are shown in Table II, where t_p represents planning time, T_f represents trajectory duration, and l_{traj} represents the trajectory length. The results indicate that, by leveraging differential flatness to simplify the state representation of the car-like robot on uneven terrain, our method achieves several-fold improvements in computational efficiency compared to Jian’s [8] and Xu’s [6] methods. Although the average length of the trajectory generated by our method is longer than that of Jian’s [8], the trajectories generated by Jian’s [8] cannot be tracked fast by the robot due to the lack of consideration for trajectory duration in its optimization model. While Xu’s [6] also incorporates T_f as an optimization variable, its inability to support reversing limits its solution space, excluding better solutions.

C. Real-world Experiments

Real-world experiments are conducted on various types of uneven terrain, including underground parking lots, grasslands, forests, farmland, and pump track, as shown in Fig. 1. These autonomous navigation tests demonstrate the effectiveness and practicality of SEB-Naver in complex real-world environments. Fig. 5 illustrates the changes in position and attitude of the robot on a pump track. More demonstrations can be found in the attached multimedia.

VIII. CONCLUSION & LIMITATIONS

In this paper, we proposed SEB-Naver, a novel $SE(2)$ -based local navigation framework for car-like robots on uneven terrain, which leverages GPU to allow real-time

TABLE II: Benchmark comparison of trajectory planner

Scenario	Method	Avg. t_p (ms)	Avg. T_f (s)	Avg. l_{traj} (m)
Mountain	Proposed	49.30	11.25	8.13
	Xu's [6]	142.39	13.88	11.64
	Jian's [8]	224.50	15.14	6.77
Forest	Proposed	57.42	12.23	8.63
	Xu's [6]	130.74	15.13	12.70
	Jian's [8]	289.22	16.65	7.22

traversability assessment for $SE(2)$ grids, introducing differential flatness for efficient trajectory optimization. Extensive simulations and real-world experiments validate its effectiveness for uneven terrain navigation. The limitation of SEB Naver lies in its reliance on accurate terrain modeling and state estimation, which may be unreliable in dynamic environments or scenarios with feature degradation. Additionally, it does not yet handle complex dynamics like slip or deformable terrain. Future work will focus on addressing these issues to improve robustness and applicability.

REFERENCES

- [1] P. Fankhauser, M. Bloesch, and M. Hutter, "Probabilistic terrain mapping for mobile robots with uncertain localization," *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 3019–3026, 2018.
- [2] T. Miki, L. Wellhausen, R. Grandia, F. Jenelten, T. Homberger, and M. Hutter, "Elevation mapping for locomotion and navigation using gpu," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022, pp. 2273–2280.
- [3] D. D. Fan, K. Otsu, Y. Kubo, A. Dixit, J. Burdick, and A. Aghamohammadi, "STEP: stochastic traversability evaluation and planning for risk-aware off-road navigation," *Robotics: Science and Systems (RSS)*, 2021.
- [4] F. Atas, G. Cielniak, and L. Grimstad, "Elevation state-space: Surfel-based navigation in uneven environments for mobile robots," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022, pp. 5715–5721.
- [5] P. Krüsi, P. Furgale, M. Bosse, and R. Siegwart, "Driving on point clouds: Motion planning, trajectory optimization, and terrain assessment in generic nonplanar environments," *Journal of Field Robotics*, vol. 34, no. 5, pp. 940–984, 2017.
- [6] L. Xu, K. Chai, Z. Han, H. Liu, C. Xu, Y. Cao, and F. Gao, "An efficient trajectory planner for car-like robots on uneven terrain," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2023, pp. 2853–2860.
- [7] M. Thoresen, N. H. Nielsen, K. Mathiassen, and K. Y. Pettersen, "Path planning for ugvs based on traversability hybrid a," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 1216–1223, 2021.
- [8] Z. Jian, Z. Lu, X. Zhou, B. Lan, A. Xiao, X. Wang, and B. Liang, "Putn: A plane-fitting based uneven terrain navigation framework," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022, pp. 7160–7166.
- [9] Z. Han, Y. Wu, T. Li, L. Zhang, L. Pei, L. Xu, C. Li, C. Ma, C. Xu, S. Shen, *et al.*, "An efficient spatial-temporal trajectory planner for autonomous vehicles in unstructured environments," *IEEE Transactions on Intelligent Transportation Systems*, 2023.
- [10] Z. Han, M. Tian, and F. Gao, "Trajectory generation for vehicle with stable time." [Online]. Available: https://mengze3.github.io/files/technical_report.pdf
- [11] K. Zhang, Y. Yang, M. Fu, and M. Wang, "Traversability assessment and trajectory planning of unmanned ground vehicles with suspension systems on rough terrain," *Sensors*, vol. 19, no. 20, p. 4372, 2019.
- [12] X. Meng, N. Hatch, A. Lambert, A. Li, N. Wagener, M. Schmitt, J. Lee, W. Yuan, Z. Chen, S. Deng, *et al.*, "Terrainnet: Visual modeling of complex terrain for high-speed, off-road navigation," *arXiv preprint arXiv:2303.15771*, 2023.
- [13] J. Frey, M. Patel, D. Atha, J. Nubert, D. Fan, A. Agha, C. Padgett, P. Spieler, M. Hutter, and S. Khattak, "Roadrunner-learning traversability estimation for autonomous off-road driving," *IEEE Transactions on Field Robotics*, 2024.
- [14] Q. Zhu, Z. Sun, S. Xia, G. Liu, K. Ma, L. Pei, Z. Gong, and C. Jin, "Learning-based traversability costmap for autonomous off-road navigation," in *China Intelligent Robotics Annual Conference*. Springer, 2024, pp. 301–312.
- [15] M. V. Gasparino, A. N. Sivakumar, Y. Liu, A. Velasquez, V. Higit, J. Rogers, H. Tran, and G. Chowdhary, "Wayfast: Traversability predictive navigation for field robots," *CoRR*, 2022.
- [16] E. Chen, C. Ho, M. Maulimov, C. Wang, and S. Scherer, "Learning-on-the-drive: Self-supervised adaptation of visual offroad traversability models," *arXiv preprint arXiv:2306.15226*, 2023.
- [17] A. J. Sathyamoorthy, K. Weerakoon, T. Guan, J. Liang, and D. Manocha, "Terrapn: Unstructured terrain navigation using online self-supervised learning," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022, pp. 7197–7204.
- [18] J. Seo, T. Kim, K. Kwak, J. Min, and I. Shim, "Scate: A scalable framework for self-supervised traversability estimation in unstructured environments," *IEEE Robotics and Automation Letters*, vol. 8, no. 2, pp. 888–895, 2023.
- [19] M. G. Castro, S. Triest, W. Wang, J. M. Gregory, F. Sanchez, J. G. Rogers, and S. Scherer, "How does it feel? self-supervised costmap learning for off-road vehicle traversability," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 931–938.
- [20] J. Seo, J. Mun, and T. Kim, "Safe navigation in unstructured environments by minimizing uncertainty in control and perception," *arXiv preprint arXiv:2306.14601*, 2023.
- [21] A. Leininger, M. Ali, H. Jardali, and L. Liu, "Gaussian process-based traversability analysis for terrain mapless navigation," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 10925–10931.
- [22] M. Ali, H. Jardali, N. Roy, and L. Liu, "Autonomous navigation, mapping and exploration with gaussian processes." *Robotics: Science and Systems (RSS)*, 2023.
- [23] H. Jardali, M. Ali, and L. Liu, "Autonomous mapless navigation on uneven terrains," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 13 227–13 233.
- [24] T. Han, A. Liu, A. Li, A. Spitzer, G. Shi, and B. Boots, "Model predictive control for aggressive driving over uneven terrain," 2024.
- [25] A. Telea, "An image inpainting technique based on the fast marching method," *Journal of graphics tools*, vol. 9, no. 1, pp. 23–34, 2004.
- [26] M. Ebrahimi and E. Lunasin, "The navier-stokes-voight model for image inpainting," *The IMA Journal of Applied Mathematics*, vol. 78, no. 5, pp. 869–894, 2013.
- [27] Z. Qiu, L. Yue, and X. Liu, "Void filling of digital elevation models with a terrain texture learning model based on generative adversarial networks," *Remote Sensing*, vol. 11, no. 23, p. 2829, 2019.
- [28] M. Stölzle, T. Miki, L. Gerdes, M. Azkarate, and M. Hutter, "Reconstructing occluded elevation information in terrain maps with self-supervised learning," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 1697–1704, 2022.
- [29] P. F. Felzenszwalb and D. P. Huttenlocher, "Distance transforms of sampled functions," *Theory of computing*, vol. 8, no. 1, pp. 415–428, 2012.
- [30] Z. Wang, X. Zhou, C. Xu, and F. Gao, "Geometrically constrained trajectory optimization for multicopters," *IEEE Transactions on Robotics*, vol. 38, no. 5, pp. 3259–3278, 2022.
- [31] C. Hertzberg, R. Wagner, U. Frese, and L. Schröder, "Integrating generic sensor fusion algorithms with sound state representations through encapsulation of manifolds," *Information Fusion*, vol. 14, no. 1, pp. 57–77, 2013.
- [32] R. T. Rockafellar, "Augmented lagrange multiplier functions and duality in nonconvex programming," *SIAM Journal on Control*, vol. 12, no. 2, pp. 268–285, 1974.
- [33] J. Reeds and L. Shepp, "Optimal paths for a car that goes both forwards and backwards," *Pacific journal of mathematics*, vol. 145, no. 2, pp. 367–393, 1990.
- [34] NVIDIA, P. Vingelmann, and F. H. Fitzek, "Cuda, release: 10.2.89," 2020. [Online]. Available: <https://developer.nvidia.com/cuda-toolkit>
- [35] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, "Fast-lid2: Fast direct lidar-inertial odometry," *IEEE Transactions on Robotics*, vol. 38, no. 4, pp. 2053–2073, 2022.